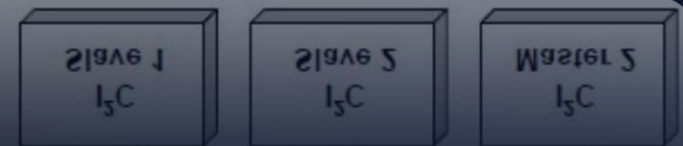
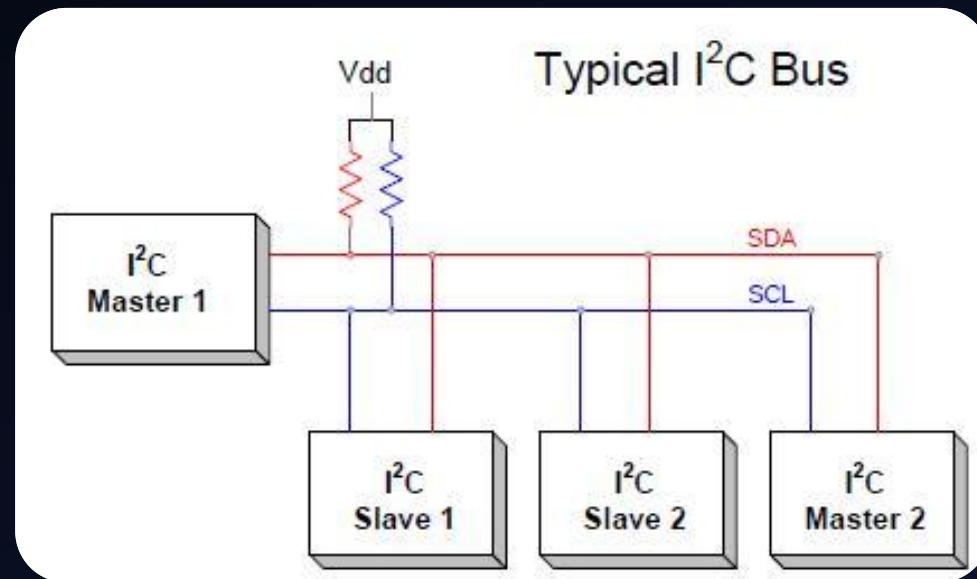


Universidad de San Carlos De Guatemala
Facultad de Ingeniería
Escuela de Ciencias y Sistemas

Organización Computacional
Conferencia del Curso
Segundo Semestre 2026

Aplicaciones del Protocolo I²C en el Desarrollo con Arduino



Sobre mi:

Kevin Estuardo Sotoj García

10mo Semestre de Ingeniería en Ciencias y Sistemas

Tutor Académico de Economía – Sección A+



3024645300102@ingenieria.usac.edu.gt



4072-2317



KevinESG97



kevin.esg

¿Qué es el I2C?

El bus de comunicaciones I2C, conocido como Inter-Integrated Circuit, es un protocolo que permite la comunicación entre múltiples dispositivos mediante solo dos hilos: uno para los datos SDA(Serial Data) y otro para el reloj SCL(Serial Clock).

La línea SDA se encarga de enviar y recibir datos, mientras que SCL sincroniza esta transferencia con pulsos de reloj.

Es ampliamente utilizado en sistemas embebidos por su simplicidad y eficacia para conectar componentes como sensores, memorias y otros periféricos.

I2C (por sus siglas en inglés *Inter-Integrated Circuit*) es un protocolo de comunicación serial desarrollado por *Phillips Semiconductors*, en el año de 1982. En un inicio se creó para poder comunicar varios chips al mismo tiempo dentro de los televisores que fabricaba la compañía. Sin embargo, con el paso del tiempo otros fabricantes comenzaron a adoptarlo hasta convertirse en el estándar del mercado mundial que es hoy.

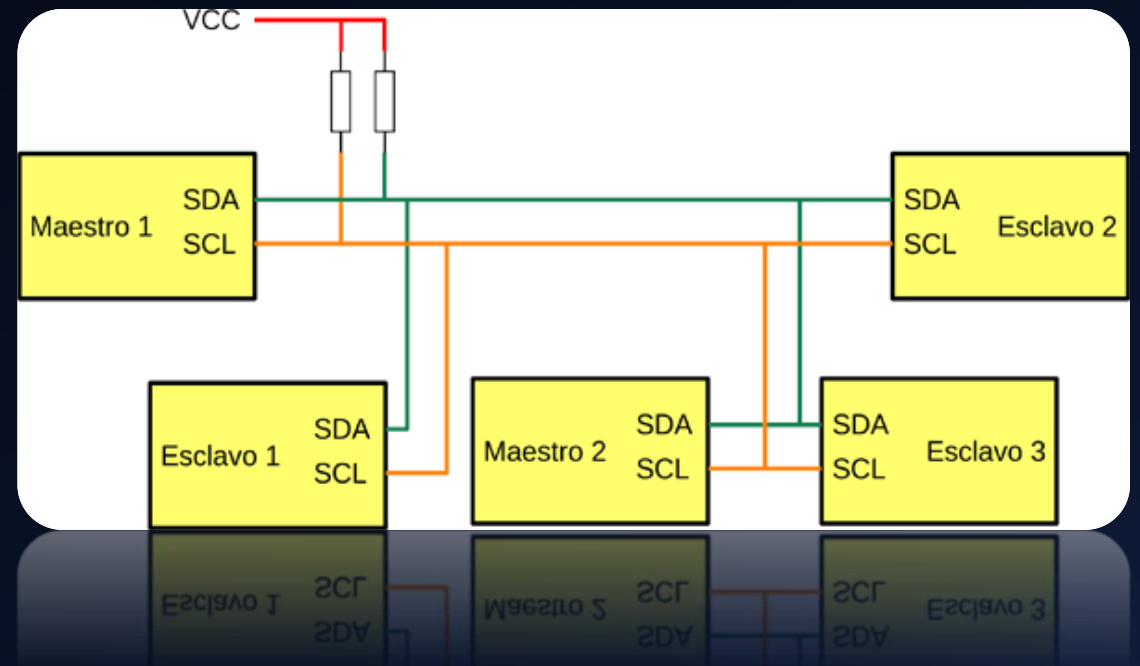
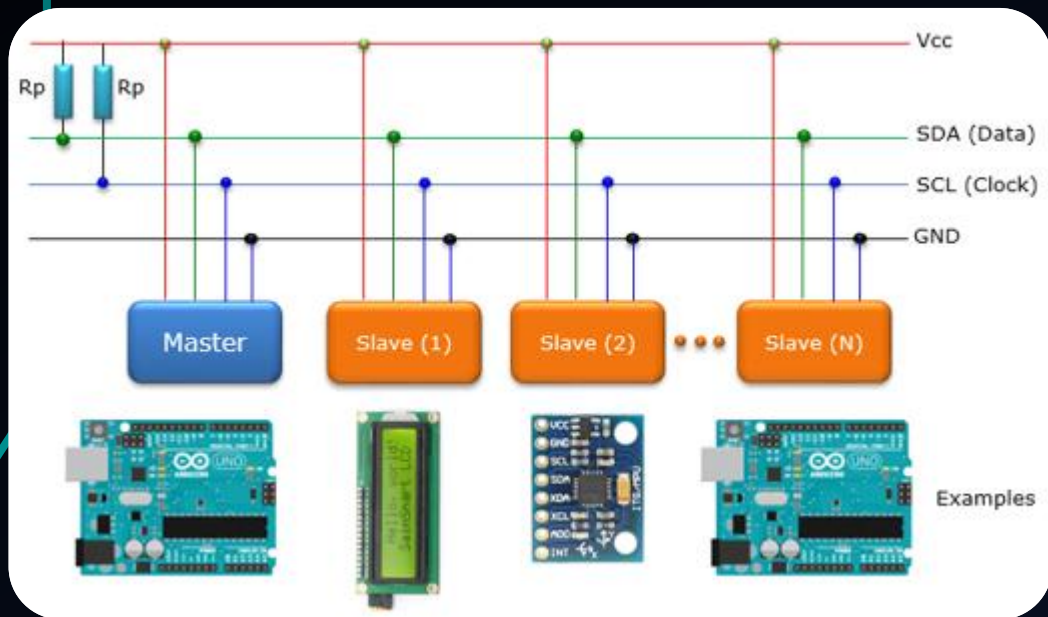


El protocolo I2C funciona con una arquitectura maestro-esclavo (*master-slave*).

En esta arquitectura existen dos tipos de dispositivos:

- Maestro (*Master*) o Controlador (*Controller*): son los que inician y coordinan la comunicación. Usualmente, cuando utilizas un Arduino en un bus I2C este es el rol que cumple.
- Esclavos (*Slave*) o Periféricos (*Peripheral*): son los dispositivos que están a la espera de que algún maestro se comunique con ellos. Casos comunes son los sensores y actuadores que soportan este protocolo, aunque también es posible, y en ocasiones necesario, que un microcontrolador se comporte como un esclavo.

Esquema de conexión de varios dispositivos que usan la comunicación i2c donde se ve que únicamente emplean esas dos líneas y así también, la alimentación VCC y GND.

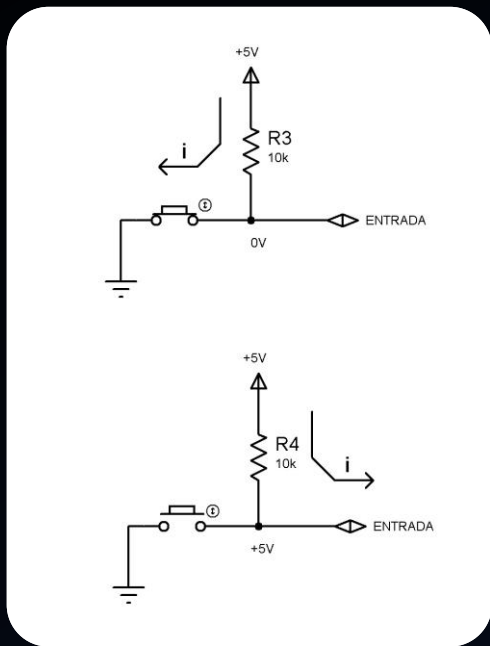




Es importante destacar las resistencia *pull-up* ubicados en ambos pines. Estas son resistencias comunes. Sin embargo, su presencia es vital para que la comunicación funcione correctamente, ya que I2C es un bus de colector-abierto.

Las resistencias pull-up y pull-down nos permiten establecer voltajes de reposo para cuando el pulsador no está presionado y asegurar una correcta lectura.

Ambas líneas SDA y SCL son líneas de colector abierto, por lo que siempre se debe colocar dos resistencias a positivo (PULLUP) para garantizar el estado alto en ambas líneas.

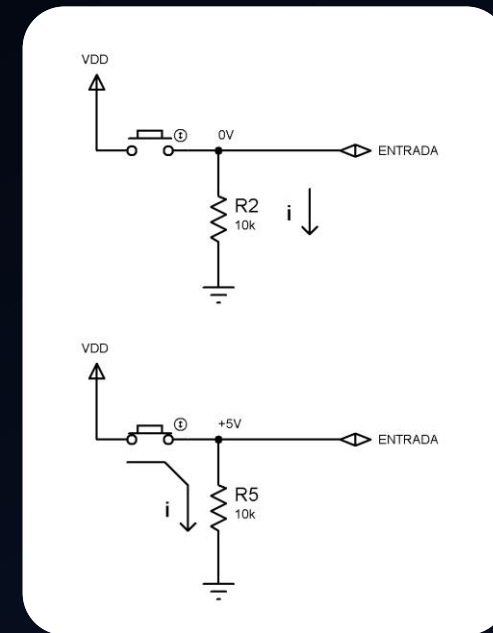


Resistencia Pull-Up

“Jala” hacia “arriba” el voltaje. De esta forma cuando el pulsador está abierto o en reposo, el voltaje en la entrada del Arduino siempre será de +5V.

Cuando el pulsador es presionado, la corriente circula por la resistencia y luego por el pulsador, de esta forma tenemos que el voltaje en la entrada del Arduino es Tierra o 0V.

Entonces en la configuración pull-up cuando el pulsador está en reposo el Arduino lee 1 y cuando presionamos leerá 0.



Resistencia pull-down

“Jala” el voltaje hacia “abajo” o “0V”. Cuando el pulsador está en reposo, el voltaje en la entrada del Arduino será 0V.

Cuando presionamos el pulsador la corriente fluye de +5V por el pulsador hacia la resistencia y termina en 0V, de esa forma tenemos +5V en la entrada del Arduino.

Entonces en la configuración pull-down cuando el pulsador está en reposo el Arduino lee 0 y cuando presionamos leerá 1.

Ejemplos en la Industria

Interfaces de sensores

- Sistemas de monitoreo ambiental que utilizan sensores de temperatura y humedad.
- Dispositivos de seguimiento de movimiento con acelerómetros y giroscopios.

Comunicación EEPROM

- Almacenamiento de ajustes de configuración en sistemas integrados.
- Guardar datos de calibración en instrumentos industriales.

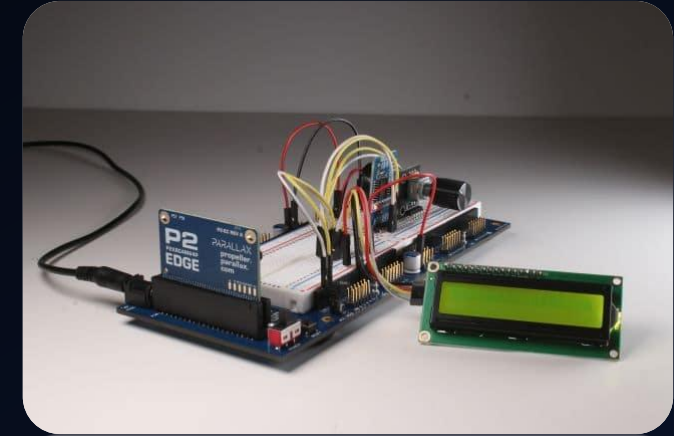


Relojes en tiempo real (RTC)

- Cronometraje en ordenadores y dispositivos IoT.
- Marcas de tiempo precisas para sistemas de registro y programación.

Interfaces de visualización

- Dispositivos portátiles con pequeñas pantallas OLED.
- Sistemas integrados que muestran información de estado en pantallas LCD.



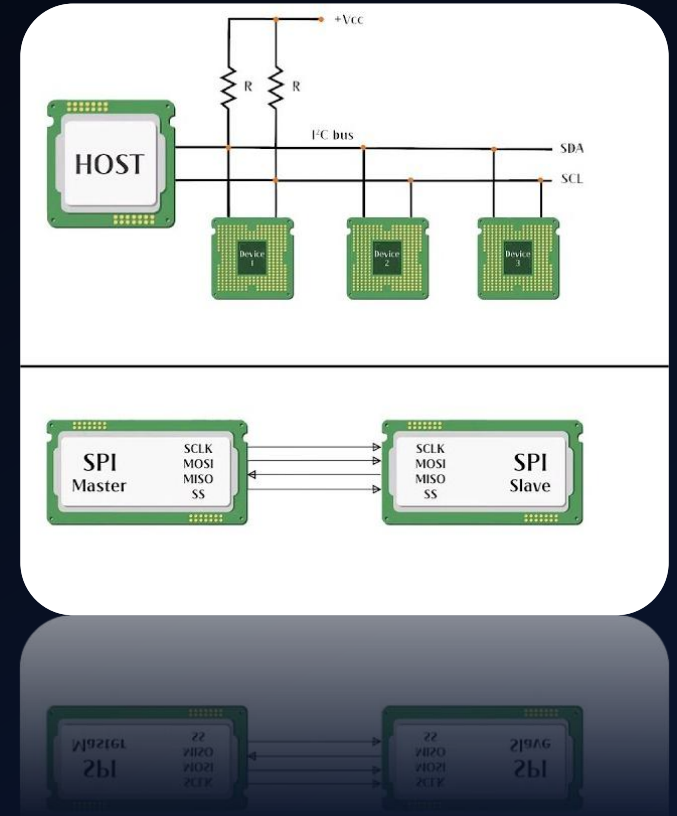
Aplicaciones industriales

- Sistemas de control de procesos que utilizan múltiples sensores y controladores.
- Monitoreo de temperatura, presión y otros parámetros de equipos de fábrica.

Observaciones Generales

¿Cuál es la diferencia entre I2C y el Protocolo SPI?

El protocolo de bus I2C utiliza dos líneas (SDA y SCL) para la comunicación con múltiples dispositivos, mientras que SPI normalmente utiliza cuatro líneas (MOSI, MISO, SCK y SS). I2C tiene direccionamiento incorporado y admite múltiples masters, lo que hace que sea más sencillo de conectar pero más lento (velocidades estándar de 100 kHz a 400 kHz). SPI es más rápido (varios MHz) pero requiere líneas de selección de chip adicionales para cada dispositivo. Para aplicaciones de IoT, a menudo se prefiere el protocolo de comunicación I2C cuando se conectan múltiples sensores debido a su cableado más simple, mientras que se elige SPI cuando se necesitan velocidades de datos más altas, como para pantallas o almacenamiento.

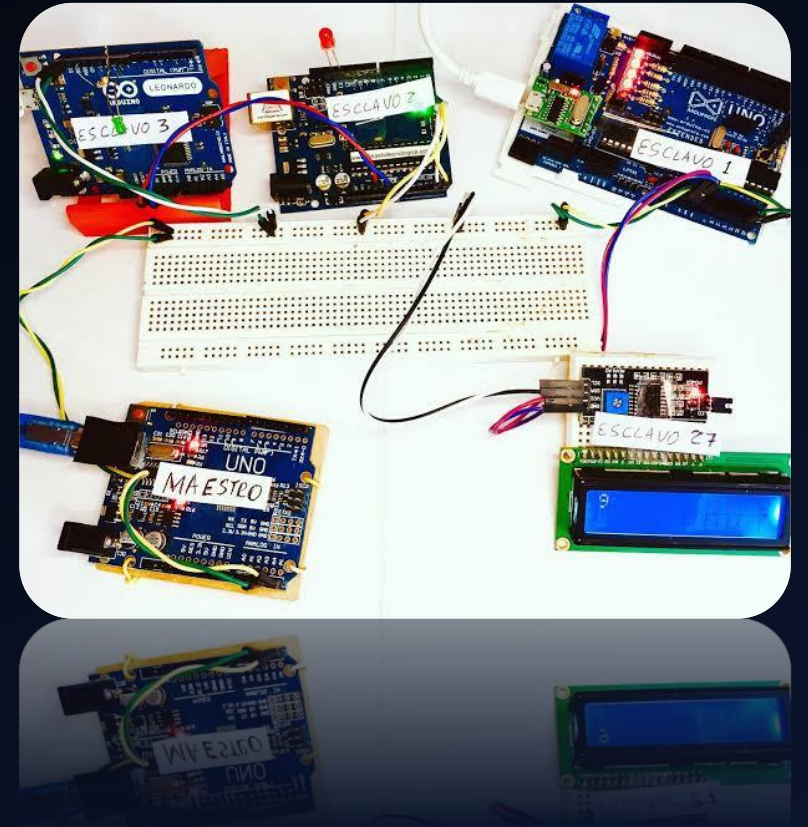


¿Cuántos esclavos se pueden conectar en el protocolo de bus I2C?

En teoría, la interfaz I2C puede admitir hasta 127 dispositivos con direccionamiento de 7 bits o 1023 dispositivos con direccionamiento de 10 bits.

Sin embargo, en aplicaciones prácticas de IoT, el límite suele ser menor debido a:

- Capacitancia del bus (limitada a aproximadamente 400 pF)
- Requisitos de energía
- Conflictos de direcciones (muchos dispositivos I2C utilizan direcciones fijas)
- Consideraciones sobre la integridad de la señal, para la mayoría de las aplicaciones de IoT, entre 8 y 12 dispositivos son un límite práctico sin utilizar multiplexores o buffers de bus.

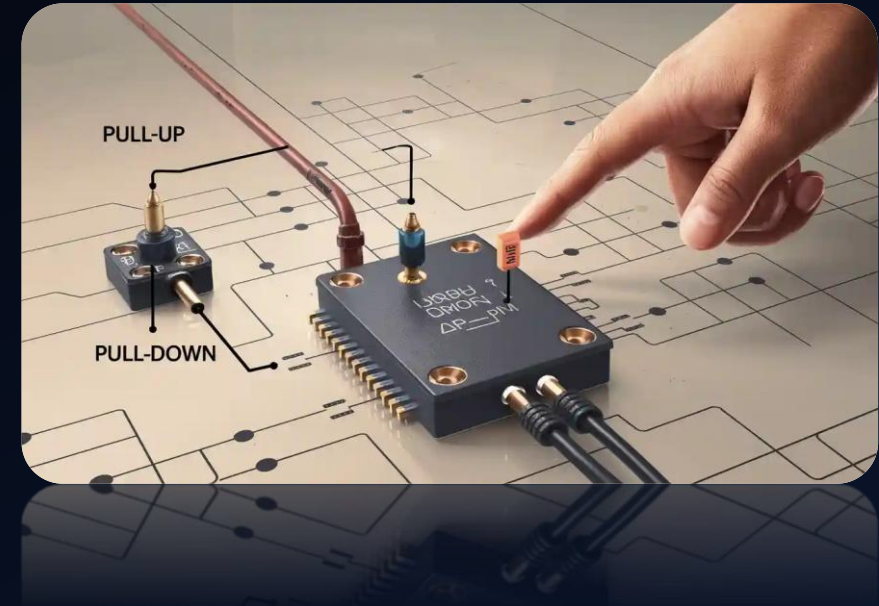


¿El protocolo I2C necesita resistencias pull-up?

El protocolo de comunicación I2C requiere resistencias pull-up en las líneas SDA y SCL porque utiliza salidas de drenaje abierto/colector abierto. Los valores de la resistencia son críticos y dependen de:

- Capacitancia del bus
- Tensión de funcionamiento
- velocidad de comunicación
- Para sistemas de 3,3 V: 2,2 k Ω a 10 k Ω
- Para sistemas de 5 V: 1,8 k Ω a 4,7 k Ω

Un valor demasiado bajo desperdicia energía, mientras que un valor demasiado alto hace que la comunicación no sea confiable. Para los dispositivos IoT, esto es crucial para el consumo de energía y la confiabilidad.



¿El protocolo de bus I2C es bueno para largas distancias?

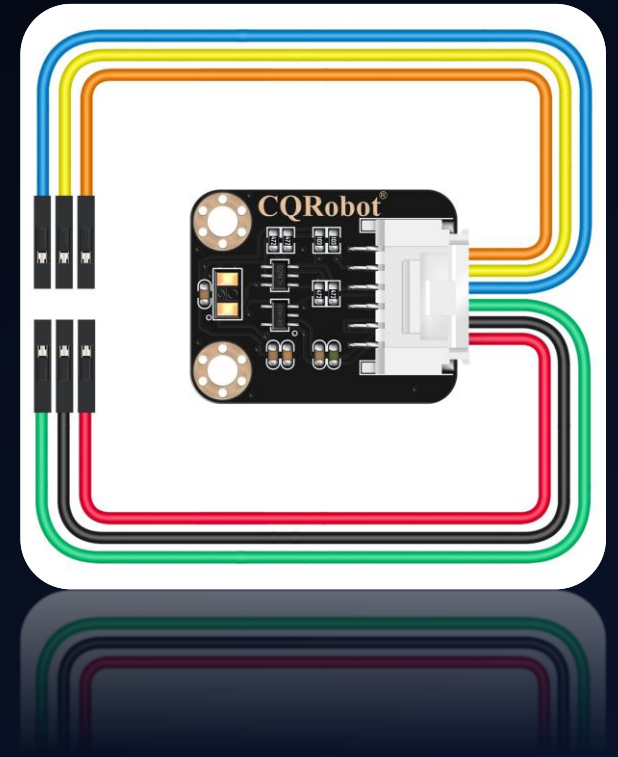
El protocolo de bus I2C fue diseñado para comunicación de corta distancia.

Límites prácticos:

- Comunicación estándar por bus I2C: normalmente fiable hasta 50 cm
- Con consideraciones especiales (amortiguadores, velocidades más bajas): hasta 2-3 metros

Las limitaciones se deben a:

- Capacitancia (400pF máx.)
 - Integridad de la señal
 - Susceptibilidad a EMI
- Para distancias más largas en aplicaciones de IoT, las soluciones incluyen:
- Tampones/repetidores I2C
 - Conversión a señalización diferencial
 - Uso de protocolos alternativos como RS-485 para largas distancias



Ejemplo práctico

